Шпаргалка для защиты лабораторных 3-7

Урбанович. Вариант 3.

Открывайте этот файл на телефоне. На компьютере держите VS Code с папкой dataMining, PDF нужной работы и браузер с n8n.

Быстрый порядок

Работа	Суть	Сохранённый результат
3	REST API, книги и авторы	360 книг; n8n 30
4	JavaScript и бесконечная прокрутка	347 товаров; n8n 30
5	Scrapy, очередь и страницы авторов	100 цитат + 304 книги; n8n 30
6	raw → clean → mart, контракт и БД	414 чистых цитат; n8n 34
7	Лимиты, ошибки, кэш и блокировки	60 архивных записей; n8n 30

Вариант 3 не означает одинаковый сайт во всех работах: предмет и источник указаны отдельно в каждом разделе. В четвёртой и пятой работах дополнительные учебные источники нужны для минимального объёма. В шестой добавлен новый API DummyJSON: в отчёте указано, какие поля есть и чего нет. HTML и сведения авторов взяты из предыдущей работы, API расширяет корпус до нужного объёма.

Перед защитой

1. Откройте в VS Code корень dataMining.

2. Откройте отчёты и блокноты нужных работ.

3. Нажмите Terminal → New Terminal. В новом терминале путь должен оканчиваться dataMining.

4. Для каждой лабы используйте отдельный терминал. Если он уже находится в labN, второй раз cd labN вводить не нужно.

5. В блокноте справа сверху выберите Select Kernel → Python Environments → Enter interpreter path. Укажите .lab-runtime/venv/Scripts/python.exe в корне проекта.

6. Блокноты показывают готовые результаты. В 3/4 оставьте RUN_COLLECTION = False.

7. В n8n проверьте обычный успешный запуск до начала показа. Для 4 нужен render_api.py, для 7 - training_server.py.

8. Ключи API для работ 3-7 не нужны. Пароль локального n8n не вставляйте в код и не показывайте преподавателю.

Если n8n не открыт

Сначала попробуйте http://localhost:5679 . Если открывается редактор или вход, сервер уже работает - второй запуск не нужен.

Если браузер пишет ERR_CONNECTION_REFUSED, из НОВОГО терминала в корне dataMining выполните:

```
$env:N8N_USER_FOLDER = "$PWD/.lab-runtime/n8n-data"
$env:N8N_RESTRICT_FILE_ACCESS_TO = "$PWD"
$env:N8N_LISTEN_ADDRESS = "127.0.0.1"
$env:N8N_PORT = "5679"
$env:N8N_RUNNERS_ENABLED = "false"
& "C:\Program Files\nodejs\node.exe" ".lab-runtime/n8n/node_modules/n8n/bin/n8n" `
  start
```

Оставьте терминал открытым. Перейдите на http://localhost:5679 . Входите в существующую локальную учётную запись; не сбрасывайте профиль и не создавайте новую базу перед защитой.

Общее для n8n

Если workflow уже есть, откройте его. Если нет: меню workflow → Import from File → labN/nocode/workflow.json. В узлах записи CSV укажите путь к нужной папке, например D:/.../dataMining/lab3/nocode/nocode_result.csv. В готовых импортированных workflow на этом ПК пути настроены.

Для показа нажмите Execute workflow и дождитесь зелёных узлов. Двойной щелчок по узлу открывает INPUT слева и OUTPUT справа; вкладка Table показывает строки. Покажите также CSV в папке лабораторной. Daily schedule - настройка расписания. Не надо активировать ежедневный запуск только ради защиты.

Успешные скриншоты лежат в labN/nocode/workflow.png и result.png. Если сеть не работает, можно показать сохранённые файлы, прямо сказав, что это предыдущий запуск.

Если времени мало

Для каждой работы: вариант → схема → код главного механизма → одна короткая команда → таблица результата → n8n → вывод. На 3 покажите заголовки и готовый анализ. На 4 - диагностику и два CSV. На 5 - JOBDIR и настройки, затем качество. На 6 - пересборку и три загрузки. На 7 - B/C и один подготовленный сбой. Не начинайте полный сбор всего каталога за несколько минут до показа.

Защита лабораторной 3

Что сделано

Мой вариант 3: Open Library Search API + Authors API. Поиск книг `python programming`, присоединение сведений первого автора, анализ языков и годов издания. Основной результат: 360 книг. n8n: 30 книг без отдельного сбора авторов. API-ключ не нужен.

1. Подготовка

В VS Code откройте dataMining, затем Terminal → New Terminal. Если терминал сейчас в другой лабе, сначала вернитесь в корень проекта. Команды ниже для этого подготовленного ПК:

```
cd lab3
$labPython = (Resolve-Path ..\.lab-runtime\venv\Scripts\python.exe).Path
```

На другом ПК установите своё окружение по README. Не копируйте путь к чужому python.exe.

Откройте report.pdf и lab3.ipynb. В блокноте выберите этот Python. RUN_COLLECTION оставьте False: ячейки покажут сохранённый набор.

2. Показать API и заголовки

Откройте collect.py: URL /search.json и параметры q, offset, limit, fields, sort. В браузере можно открыть https://openlibrary.org/search.json?q=python&limit=1 .

Для настоящего нового запроса и вывода заголовков:

```
& $labPython collect.py --show-headers
```

Что показать в терминале и headers_example.json:

- GET и URL с параметрами.

- User-Agent: какая программа обращается к API.

- Accept: ожидается JSON.

- Статус 200 при успешном ответе.

- Content-Type: формат ответа.

- RateLimit-* показывать только если сервер их прислал. Их отсутствие нормально.

- Retry-After учитывается при 429; при обычном 200 его может не быть.

Фраза: «Отправляю GET с параметрами поиска. Проверяю статус и формат, затем читаю JSON».

Если сеть сейчас отказывает, покажите сохранённый headers_example.json и сообщите, что это предыдущий ответ. Не называйте его новым запуском.

3. Пагинация, повторы и авторы

В collect.py найдите paginate и get_json. Четыре порции по 90: offset 0, 90, 180, 270. Пауза 1,1 с, timeout 20 с; 429 и 5xx повторяются максимум дважды после первой попытки. Покажите collection_log.csv и authors_cache.json.

Фраза: «Один ответ не содержит весь набор. Offset сдвигается на размер полученной порции. Авторов запрашиваю по ключу и кэширую».

4. Показать результат и качество

Выполните ячейки lab3.ipynb сверху вниз при RUN_COLLECTION = False. Покажите таблицу первых строк, полноту, годы издания и языки.

Нужные файлы: data_clean.csv, completeness.csv, quality.json, analysis_summary.json. Ожидается: 360 уникальных книг, ноль дублей и пустых обязательных полей; 357 книг с найденными сведениями автора. 225 строк в book_languages.csv - связи книга-язык, а не число книг. У 216 книг известен хотя бы один язык, у 207 есть английский.

Фраза: «Связываю таблицы по author_key. Языков у книги может быть несколько, поэтому они вынесены в отдельную таблицу».

5. Короткий повторный сбор

Чтобы сохранить исходную выборку отчёта, сделайте отдельную папку с копией состояния:

```
$demoDir = "demo_" + [guid]::NewGuid().ToString("N").Substring(0,8)
New-Item -ItemType Directory -Path $demoDir | Out-Null
Copy-Item -LiteralPath data_raw.jsonl,state.json,authors_cache.json -Destination `
  $demoDir
& $labPython collect.py --output $demoDir --run-label demo
```

Смотрите new_records и число запросов в metrics_demo.json внутри новой папки. Раньше повтор дал ноль новых ключей; API может измениться, поэтому обещать ноль заранее нельзя. Полный новый сбор с авторами

занимает несколько минут и для короткого показа не нужен.

6. Показать n8n

Откройте http://localhost:5679 и workflow «Lab 3 Open Library REST API». Если его нет: меню workflow → Import from File → lab3/nocode/workflow.json. При импорте проверьте абсолютные пути Save CSV / Save error log к lab3/nocode. Готовая локальная установка n8n уже настроена; не создавайте второй профиль.

1. Fetch books: GET, параметры, три порции по 10, интервал 1,1 с.

2. Check books response: проверка ответа и ветка ошибки.

3. Transform fields: названия полей и числовые типы.

4. Daily schedule: ежедневное расписание.

5. Execute workflow. Дождитесь зелёных узлов.

6. Откройте выход Transform fields / CSV: 30 строк.

7. Откройте nocode/nocode_result.csv.

Фраза: «Это уменьшенный сбор без Authors API. С Python сравниваю пять общих полей; все 30 строк совпали».

7. Завершение и частые вопросы

Покажите таблицу сравнения в отчёте. Нельзя сравнивать только полное время 360 книг с авторами и 30 книг без авторов: объём разный.

- JSONL хранит исходные записи; CSV удобен для таблицы.

- Кэш уменьшает запросы к авторам; известные ключи не добавляются второй раз.

- Годы и языки характеризуют эту выборку, а не весь каталог.

- Публичное имя автора может быть персональными данными; контакты и биографии здесь не нужны.

Дополнительно покажите outliers.json: 11 необычных лет и 50 значений edition_count по IQR; строки сохранены, потому что бизнес-правила не нарушены.

Защита лабораторной 4

Что сделано

Мой вариант 3: бесконечная прокрутка ScrapingCourse. На назначенном сайте найдено 147 товаров. Для минимума 200 добавлен дополнительный ScrapingSandbox с 200 товарами. Итого 347. Сравниваются обычный браузер и сбор с меньшим числом загрузок.

1. Подготовка

Откройте новый терминал в корне dataMining:

```
cd lab4
$labPython = (Resolve-Path ..\.lab-runtime\venv\Scripts\python.exe).Path
```

Откройте report.pdf, lab4.ipynb и collect.py. Блокнот по умолчанию читает сохранённые результаты; RUN_COLLECTION = False. Для сборки на другом ПК сначала выполните установку и playwright install chromium по README.

2. Объяснить, почему нужен браузер

В обычном браузере откройте https://www.scrapingcourse.com/infinite-scrolling . Прокрутите вниз: карточки догружаются.

Откройте в браузере F12 → Network → Fetch/XHR, включите Preserve log и прокрутите снова. Покажите запрос /ajax/products и его страницу/порцию. Название запроса видно в сохранённом collection_log.csv. У ScrapingSandbox отдельный полный XHR-источник в этой проверке не найден; там остаётся облегчённый браузер.

Можно повторить диагностику:

```
& $labPython collect.py --mode diagnose
```

В artifacts/diagnostics.json: в исходном HTML 12 и 20 карточек. Без JavaScript прокрутка не увеличивает эти числа. Фраза: «HTTP получает только первоначальную разметку. Остальные товары добавляет JavaScript».

3. Показать браузерный сбор

В collect.py найдите scroll_collect, wait_for_function, eval_on_selector_all и route. Если название функции отличается, ищите указанные вызовы.

- Локаторы выбирают карточки и поля по классам.

- Прокрутка ждёт увеличения числа карточек, а не только фиксированного sleep.

- Остановка: нет роста, достигнут предел или вышел общий бюджет.

- eval_on_selector_all извлекает все карточки одной операцией.

- Изображения, шрифты и медиа можно не скачивать.

- При сбое сохраняются HTML и скриншот.

При необходимости запустите весь сбор (обычно около минуты, зависит от сайтов):

```
& $labPython collect.py --mode all
```

Команда обновит результаты этой лабы. Для короткой защиты достаточно сохранённого запуска и диагностики.

4. Результаты и сравнение

Выполните ячейки lab4.ipynb сверху вниз. Покажите data_browser.csv, data_light.csv и comparison.json. Ожидается по 347 строк и совпадение ключей, названий, цен и URL.

Откройте metrics_browser.json и metrics_light.json. В отчёте сохранены замеры около 57 и 27 с; новый запуск может дать другое время. Облегчённый путь: XHR у ScrapingCourse; браузер без тяжёлых ресурсов у Sandbox. Не говорите, что оба источника собраны без браузера.

Фраза: «Проверяю одинаковый результат. Потом сравниваю время и объём ответов, а не только скорость».

5. n8n и рендерер

В отдельном терминале из lab4 запустите:

```
$labPython = (Resolve-Path ..\.lab-runtime\venv\Scripts\python.exe).Path
& $labPython render_api.py --port 8789
```

Оставьте окно открытым. Рендерер сам запускает браузер, прокручивает страницу и отдаёт готовый HTML. Это Python-сервис перед n8n; n8n сам JavaScript страницы не выполняет.

Откройте http://localhost:5679 и workflow «Lab 4 Dynamic products via rendering API». При необходимости импортируйте lab4/nocode/workflow.json. Проверьте абсолютные пути Save CSV / Save error log.

1. Render page with JS: URL http://127.0.0.1:8789/render .

2. Check rendered HTML: проверка ответа.

3. Узел извлечения HTML: правила поиска полей.

4. Daily schedule: расписание.

5. Execute workflow.

6. Откройте результат и nocode/nocode_result.csv: 30 товаров.

Если ECONNREFUSED на 8789 - сначала запустите render_api.py. Если 5679 не открывается - нужен запущенный n8n.

6. Что сказать в конце

«Для динамической страницы нужно дождаться появления данных. Если есть подходящий XHR, он дешевле браузера. Если его нет, оставляю браузер, но отключаю лишние ресурсы. В обоих случаях проверяю полноту и одинаковые ключи».

Покажите в отчёте диагностику, таблицу двух подходов, качество и скриншоты n8n.

Покажите artifacts/smoke_check.json: набор проходит, пустой и неполный случаи дают предупреждение. Для Sandbox блокировка медиа уменьшила запросы 215 → 22, но не доказала экономию тел ответов.

Дополнительный замер load/domcontentloaded/commit на одинаковых 200 товарах сохранён в artifacts/browser_optimization.json. В облегчённом режиме выбран commit; затем явно ожидаются карточки.

Защита лабораторной 5

Что сделано

Мой вариант 3: цитаты и страницы авторов Quotes to Scrape. Нужно показать краулер: страницы каталога → карточки → страницы авторов. На Quotes всего 100 цитат. Для минимума 300 добавлен дополнительный Books to Scrape: 304 книги. В БД 404 записи двух видов; это не 404 цитаты.

1. Подготовка и обзор

Новый терминал из корня dataMining:

```
cd lab5
$labPython = (Resolve-Path ..\.lab-runtime\venv\Scripts\python.exe).Path
```

Откройте report.pdf, labcrawler/spiders/quotes.py, pipelines.py и settings.py. В quality/quality_report.ipynb выберите подготовленный Python и выполните ячейки сверху вниз: он читает сохранённые данные.

Фраза: «Scrapy управляет очередью URL и вызывает callbacks. Паук извлекает поля, pipelines проверяют и сохраняют записи».

2. Показать паука

В quotes.py найдите parse, ссылку next и запрос страницы автора. Покажите передачу цитаты через cb_kwargs и обработку ответа автора. Укажите allowed_domains и ограничения глубины/времени.

Схема: каталог → цитата и ссылка автора → страница автора → дополненная запись. Авторы кэшируются; одна и та же страница не должна скачиваться для каждой цитаты.

3. Показать обработку и настройки

В pipelines.py идут валидация → очистка → дедупликация → SQLite. Цитата определяется текстом и автором, книга - UPC. SQLite получает записи пакетами; обязательные пропуски уходят в DropItem.

В settings.py покажите: ROBOTSTXT_OBEY, USER_AGENT, DOWNLOAD_DELAY, AUTOTHROTTLE, RETRY_TIMES, DOWNLOAD_TIMEOUT, DEPTH_LIMIT, HTTPCACHE и ITEM_PIPELINES. Рядом оставлены короткие комментарии, требуемые заданием.

4. Остановка и продолжение

Для каждого показа нужен НОВЫЙ каталог JOBDIR. Команды сами создают уникальное имя:

```
$demoId = [guid]::NewGuid().ToString("N").Substring(0,8)
$jobDir = "jobstate/demo_$demoId"
& $labPython -m scrapy crawl quotes -a "run_id=pause_$demoId" -s `
  "JOBDIR=$jobDir" -s CLOSESPIDER_ITEMCOUNT=35 -O "data/pause_$demoId.jsonl"
& $labPython -m scrapy crawl quotes -a "run_id=resume_$demoId" -s `
  "JOBDIR=$jobDir" -s CLOSESPIDER_ITEMCOUNT=300 -O "data/resume_$demoId.jsonl"
```

В первом логе покажите item_scraped_count и причину закрытия. Во втором - продолжение, dupefilter и итоговый объём. В проверенном запуске было 35 + 65 = 100 цитат. Граница остановки может немного превыситься из-за запросов в работе.

JOBDIR хранит очередь и посещённые URL. Для продолжения используется тот же путь. Не запускайте два паука одновременно с одним JOBDIR.

5. Кэш и качество

Покажите сохранённые data/stats_cache_1.json и data/stats_cache_2.json: в тёплом запуске 211 cache hit и существенно меньше время. JOBDIR и HTTP-кэш решают разные задачи: очередь и копии ответов.

Для нового сравнения кэша (дольше, необязательно на короткой защите):

```
$cacheDir = "demo_cache_" + [guid]::NewGuid().ToString("N").Substring(0,8)
& $labPython -m scrapy crawl quotes -a run_id=cold_demo -s `
  "HTTPCACHE_DIR=$cacheDir" -O data/cold_demo.jsonl
& $labPython -m scrapy crawl quotes -a run_id=warm_demo -s `
  "HTTPCACHE_DIR=$cacheDir" -O data/warm_demo.jsonl
```

Качество:

```
& $labPython quality/analyze.py
```

Покажите quality/current.json и таблицы блокнота: 100 цитат, 304 книги, ноль дублей и пустых обязательных полей. В БД откройте таблицы books и quotes; если расширения SQLite нет, достаточно таблиц блокнота. Контроль деградации находится в quality/smoke_check.py: падение числа строк >20% или обязательные пропуски >5% дают код 1.

6. n8n

Откройте http://localhost:5679 и workflow «Lab 5 Two level quote collection». При необходимости импортируйте lab5/nocode/workflow.json и проверьте абсолютные пути сохранения.

1. Узел загрузки страниц: три страницы по десять цитат.

2. Узел перехода к автору: второй уровень обхода.

3. Дедупликация по ключу цитаты.

4. Расписание, таймаут и ветка ошибки.

5. Execute workflow.

6. Выходная таблица и nocode/nocode_result.csv: 30 цитат.

Фраза: «Здесь тоже два уровня. В Scrapy есть постоянная очередь и JOBDIR; в этом небольшом n8n-сценарии их нет».

7. Завершение

Покажите отчёт: структура пауков, настройки, остановка/продолжение, качество, сравнение и скриншоты. Книги добавлены только для объёма; основной предмет варианта - цитаты и авторы.

Ветка ошибки n8n теперь заканчивается Stop and notify после сохранения CSV. Подготовленный 404 проверен; подтверждение - nocode/metrics_error_demo.json.

Защита лабораторной 6

Что сделано

Мой вариант 3: цитаты и авторы. Это работа про хранение и качество уже собранных данных. HTML Quotes содержит теги, учебный API DummyJSON - текст и автора без тегов. API добавлен для объёма от 300. Из 452 строк получены 414 чистых цитат. 32 входные строки (7,08%) намеренно испорчены и помечены; это разрешённая демонстрация очистки.

1. Подготовка и слои

Новый терминал из корня dataMining:

```
cd lab6
$labPython = (Resolve-Path ..\.lab-runtime\venv\Scripts\python.exe).Path
```

Откройте report.pdf и quality_report.ipynb. Покажите папки raw, clean, mart. raw - исходные значения и происхождение; clean - исправленные и проверенные строки; mart - агрегаты для анализа.

В raw/html_quotes.jsonl покажите одну строку: _source, _source_url, _collected_at, _parser_version, _status и сырые поля. Manifest хранит контрольные суммы прежних байтов. Старые строки не переписываются.

2. Пересобрать без сайтов

```
& $labPython -m src.rebuild
```

В терминале и clean/rebuild_summary.json: raw_rows 452, clean_rows 414, authors 98, tag_links 205, raw_unchanged true. Если clean уже загружен, load.updated должен быть 0. Время и возраст данных при повторе могут измениться.

Фраза: «Для повторной очистки сайт не нужен. Беру raw, применяю тот же алгоритм и проверяю контрольные суммы».

В mart/snapshot_validation.json две пересборки: по 414 строк и 414 уникальных ключей. Это показывает, что партиции не накапливают дубли.

3. Форматы и очистка

Выполните ячейки quality_report.ipynb сверху вниз. Покажите таблицу CSV / JSONL / Parquet: размер, полное чтение, чтение трёх столбцов. В примере ID 000100 автоматическое определение типа в CSV/JSONL теряет нули; явный dtype исправляет чтение. Parquet сохраняет строковый тип и дату UTC.

Покажите clean/cleaning_protocol.csv: 452 → 452 → 452 → 446 → 434 → 420 → 414.

Порядок: текст → ключ/URL → типы → пропуски → дубли → бизнес-правила. 24 строки отбракованы, 14 дублей удалены. Неизвестный год рождения остаётся NULL; теги из API не выдумываются.

4. Контракт и брак

Откройте src/contract.py: модель Quote, field_validator и model_validator. Покажите clean/contract_examples.json или таблицу в блокноте: пустой текст, короткое имя, неверный тег, дата без зоны, неверный ключ и другие ошибки отклонены. clean/rejects.jsonl содержит причину, поле и исходную запись.

Фраза: «Контракт проверяет типы, диапазоны и согласованность. Например, ключ обязан соответствовать тексту и автору».

5. БД, три загрузки и история

Откройте db/schema.sql: quotes, authors, quote_tags, rejects, quote_history. Покажите PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK, UNIQUE и индексы.

```
& $labPython -m db.load --demonstrate
```

Демонстрация пересоздаёт отдельную idempotency_demo.db, основную warehouse.db не меняет. В clean/idempotency.json:

- A: вставлено 414, история 414.

- B: вставлено 0, обновлено 0, история 414; updated_at не изменён.

- C: обновлены ровно две строки, история 416.

- Проверки duplicate_key, check, not_null: rejected.

Фраза: «Идемпотентность - повтор одинаковых данных не меняет состояние. Изменения двух записей фиксируются отдельно».

6. Светофор и mart

Покажите clean/quality.json: восемь метрик, пороги и green. У необязательного года рождения полнота ниже, чем у текста и автора; это разные требования. Валидность считается до удаления дублей: 428 из 452 строк прошли контракт, затем осталось 414.

Проверка запрета плохой выгрузки:

```
& $labPython -m src.rebuild --force-red
$LASTEXITCODE
```

Ожидаемый код 2 - успешная демонстрация защиты. Mart при этом не перезаписывается. Затем покажите три файла mart/views и чтение одной партиции в блокноте. Два графика: источники и частые теги.

7. n8n

Откройте http://localhost:5679 и workflow лабораторной 6. При необходимости импортируйте lab6/nocode/workflow.json. Проверьте абсолютные пути Read source snapshot, Save CSV, Save rejects. Источник - lab6/nocode/input.json.

1. Normalize fields: очистка текста, автора, даты и ID.

2. Crypto: ключ цитаты.

3. IF: валидные и невалидные записи.

4. Remove duplicates: дубли.

5. Execute workflow.

6. В выходе Select output fields должно быть 34 строки; в rejects - три.

7. Покажите оба CSV и Daily schedule.

Вход39 = выход34 + брак3 + дубли2. CSV создаётся самим n8n. Выбранный CSV не умеет UPSERT и историю; для этого нужна другая ветка с БД. Code-узел разрешён в задании, это не скрипт очистки после выгрузки.

8. Завершение

Покажите в отчёте протокол, три загрузки, светофор и сравнение с n8n. Имена авторов и год рождения рассматриваются как потенциальные ПД; контакты и биографии не нужны. Не утверждайте, что публичность цитаты автоматически разрешает любое использование.

Защита лабораторной 7

Что сделано

Мой вариант 3: HTML + API; 500 у одного источника, другой работает; 404 на отдельных URL. Работа про устойчивость HTTP-сборщика, а не про новый большой набор. Профили A/B/C проверяются локально на одинаковых 60 архивных записях: 30 цитат и 30 книг. Так можно воспроизвести сбой, не нагружая внешний сайт.

1. Запустить учебный сервер

Новый терминал из корня dataMining:

```
cd lab7
$labPython = (Resolve-Path ..\.lab-runtime\venv\Scripts\python.exe).Path
& $labPython training_server.py
```

Оставьте окно открытым. Серверы работают на 127.0.0.1:8797 и 127.0.0.2:8798, только на этом ПК. Если порт занят уже работающим учебным сервером, не запускайте второй экземпляр.

Во втором терминале из корня:

```
cd lab7
$labPython = (Resolve-Path ..\.lab-runtime\venv\Scripts\python.exe).Path
```

Откройте report.pdf и analysis.ipynb. Блокнот читает отчёты запусков; выберите этот Python и выполните ячейки сверху вниз.

2. Показать ограничитель и повторы

Откройте src/limiter.py: token bucket, burst 1, два запроса/с на origin, jitter, адаптация. Не путайте число воркеров с числом одновременных запросов на один домен. Для каждого домена один владелец; файловая блокировка запрещает второму процессу одновременно забрать тот же домен.

В src/retry.py: 429, 5xx и сетевые ошибки можно повторять; обычные 4xx нельзя. Retry-After важнее собственного backoff; поддерживаются секунды и HTTP-дата. Максимум три попытки, бюджет восемь секунд на URL. Если Retry-After не помещается в бюджет, URL прекращается, а не запрашивается раньше срока.

3. Сравнить A/B/C

```
& $labPython -m src.collect --profile A
& $labPython -m src.collect --profile B
& $labPython -m src.collect --profile C
```

A - без ограничений, только локально. B - с ограничителем и повторами, заполняет кэш. C - плюс conditional GET, gzip и watermark.

Откройте reports/run_report_A.json, _B.json, _C.json. Ожидаемые свойства (время плавает):

- Во всех результатах 60 записей.

- A/B: по десять запросов и 60 полученных по сети записей.

- Повторный C: четыре 304, шесть пропущенных страниц, ноль новых записей по сети.

- C возвращает 60 записей из проверенного архива, а не 60 новых загрузок.

- wire_bytes считает тела ответов. У 304 тело пустое, но HTTP-заголовки всё равно передаются.

В state/etags.json покажите ETag/Last-Modified; в watermark.json - версии ресурсов. В reports/load_savings.json отдельно измерены conditional, gzip и incremental. Для свежих графиков после команд повторно выполните ячейки analysis.ipynb.

4. Сбой одного источника и 404

Перед повтором сбойных сценариев перезапустите учебный сервер: Ctrl+C в первом окне и снова training_server.py. Это сбросит счётчики намеренных ошибок.

```
& $labPython -m src.collect --scenario faults
```

Откройте reports/scenario_faults.json: initial_html_failed true, api_continues true, html_recovered true, final_breaker_state closed. В указанном там JSONL-журнале найдите circuit_open, skip_open, circuit_half_open и circuit_closed. Два 404 запрошены по одному разу; 429 проверен с Retry-After в секундах, датой и без него.

Фраза: «После двух 500 домен временно выключается. Другой источник работает. Затем делается один пробный запрос». Красный статус здесь ожидаем: ошибки подготовлены специально.

5. Заглушка с HTTP 200

```
& $labPython -m src.collect --scenario blocked
$LASTEXITCODE
```

Ожидается код 2. В reports/scenario_blocked.json: HTML остановлен, API продолжает. В logs/incidents сохранены тело и заголовки. Откройте src/detector.py: проверяются не только коды, но и маркеры CAPTCHA, размер тела и наличие нужных элементов.

Фраза: «HTTP 200 ещё не означает, что пришли данные. При заглушке останавливаю источник и сохраняю причину».

6. Логи, пороги и настоящие сайты

В logs/run_*.jsonl одна строка - JSON-объект со статусом, временем, попыткой, байтами и ошибкой. В analysis.ipynb покажите коды, медиану/p95, повторы и два графика. Порогов шесть: ошибки, 429/403, p95, падение объёма, уменьшение тела и блокировка.

Откройте reports/real_probe.json. В сохранённой реальной проверке Quotes дал 10 цитат; Open Library Search был пропущен из-за robots.txt. Это запрет правилами, а не доказательство IP-блокировки. Не обходите его и не выдавайте локальные 60 записей за новый ответ внешнего API.

7. n8n

Учебный сервер должен работать. Откройте http://localhost:5679 и workflow лабораторной 7. При необходимости импортируйте lab7/nocode/workflow.json и проверьте абсолютные пути Save CSV / Save error log.

1. Prepare requests: три страницы API.

2. Loop requests: по одной странице.

3. Wait: пауза 0,6 с.

4. HTTP Request: timeout, Retry On Fail для сетевых ошибок, полный HTTP-ответ.

5. Classify response: ограниченные повторы 429/5xx; 404 не повторяется.

6. Проверка числа строк и ветка ошибки → Stop and Error.

7. Execute workflow: три запроса и 30 строк, выходной CSV.

8. Daily schedule: расписание.

Покажите nocode/metrics_rate_limit.json: у подготовленного 429 четыре запроса вместо трёх. nocode/metrics_missing_url.json: 404 отправлен один раз, workflow завершён ошибкой. Это сохранённые отдельные проверки, обычный workflow сейчас должен пройти зелёным.

Нативного предохранителя с изоляцией доменов и conditional-кэша в этом workflow нет. Ежедневный обычный запуск этого подмножества: 30 исполнений и 90 HTTP-запросов за 30 дней. Локальный n8n и API без платы; оборудование и электричество отдельно не оценивались.

8. Завершение

Покажите сравнение A/B/C, экономию трёх механизмов и legal_checklist.md. «Ограничитель делает сбор медленнее, зато контролирует нагрузку. Кэш уменьшает запросы и трафик. При блокировке я прекращаю сбор».

Покажите reports/rate_limits.json: минимальные интервалы выше 0,5 с. В faults сработали пороги ошибок и 429/403, у заглушки - порог заблокированных доменов.